

MiniPoker Project: Strategic Agent Design and Tournament Play

Due Date: May 1, Friday at 11:59pm.

Overview

In this project, you will design and implement a strategy for a simplified two-player poker-like game. The core objective is to develop an intelligent agent capable of playing the game competitively. At the end of the project, your agent will compete against your classmates' agents in a round-robin tournament.

Game Description

The MiniPoker game is a simplified two-player, two-stage poker variant with partial information and strategic betting. The game unfolds as follows:

- Each player is dealt one private card from a shuffled deck consisting of 10 cards: A, K, Q, J, T in both **Hearts** and **Spades**.
- The game consists of two stages of betting. After the first stage, a *shared table card* is drawn and revealed to both players.
- The final hand for each player is formed from their private card and the table card. Rankings are determined as follows:

Pair > Straight Flush > Straight > Color (Same Suit) > High Card

- The game ends when one player folds, or both players call/check through both stages. The winner is determined based on hand rankings.

Stage 1 (Pre-Flop)

- **Player 1 (P1)** opens the game. They may:
 - C: Check (pass the move to P2)
 - R1: Raise a small amount
 - R2: Raise a large amount
- **Player 2 (P2)** responds based on P1's action:
 - If P1 checks: P2 may C (check), R1, R2, or F (fold)
 - If P1 raises: P2 may C1 (call small), C2 (call large), or F
- If a raise is made, P1 may respond by C1/C2 (if appropriate) or F, but cannot re-raise.

If no one folds, a **table card** is drawn and revealed. The game proceeds to Stage 2.

Stage 2 (Post-Flop)

- **Player 1** acts first. They may:
 - C: Check
 - R1: Raise small
 - R2: Raise large
- **Player 2** then responds:
 - If P1 checks: P2 may C, R1, R2
 - If P1 raises: P2 may C1/C2 or F
- If P2 raises: P1 may only C1/C2 or F (no re-raises allowed)

Payoff: If a player folds, the other player automatically wins. If the hand reaches resolution, payoffs depend on the hand comparison and the last call made (C, C1, or C2), as detailed in the game engine.

Starter Code and Agents

Starter code is provided, including the main game engine and three baseline agents:

- **RandomAgent:** Selects actions randomly among allowed options.
- **AggressiveAgent:** Prefers raises and calls whenever possible.
- **PassiveAgent:** Tends to check or fold (against raises).

You are encouraged to:

- Read through and experiment with the codebase.
- Simulate rounds using the provided agents to understand game dynamics and history structure.
- Modify or extend the test framework for debugging and training your agent.

Your Task

Your main objective is to design your own poker-playing agent. Your agent should:

- Decide on a strategy based on the private card, table card (if revealed), and game history.
- Handle both stages of the game according to the valid action sets.
- Be robust to different opponent types.

Submission Requirements

- Your agent must be a Python class inheriting from `BaseAgent` like the other example agents provided (see for instance the `AggressiveAgent`, `PassiveAgent`, and `RandomAgent` classes)
- Your class must implement the method `act(self, card, table_card, stage, history)`.
- Submit your Python file named `your_groupname_agent.py`.
- Your code should be commented to clearly describe what it is supposed to be doing.
- Include a project report (at most 4 pages) that includes a clear description of your agent, the performance estimates, and other relevant information.

Tournament and Evaluation

All submitted agents will compete in a round-robin tournament.

- Each agent will play multiple (100) rounds against each other agent.
- Performance will be measured by cumulative win/loss margin.
- Bonus recognition may be given to top-performing strategies.

Grading Rubric

- 40% Correct implementation and valid actions
- 30% Project report
- 20% Code clarity and modularity
- 15% Tournament performance

Getting Started

We recommend beginning with:

- Exploring the `MiniPokerGame` class
- Running the test agents against each other
- Modifying and logging behaviors to understand how the game unfolds

You are free to choose your design approach:

- Rule-based strategy (e.g., always raise with high cards, fold with weak combinations)

- Heuristic methods
- Belief tracking / opponent modeling (Fictitious Play, Bayesian or some other version)
- Online/Reinforcement learning

Good luck — and may your agent play smart and win big!